

Rapport – LOG430 Labo 02

Thomas Journault

Question 1

Lorsque l'application démarre, la synchronisation entre Redis et MySQL est-elle initialement déclenchée par quelle méthode ?

Au démarrage, `store_manager.py` appelle `populate_redis_from_mysql()`, qui délègue à `sync_all_orders_to_redis()`. Cette dernière vérifie si Redis contient déjà des commandes (via `r.keys("order:*")`). Si Redis est vide, elle charge toutes les commandes depuis MySQL et les insère dans Redis. Ce mécanisme garantit que la synchronisation ne s'exécute qu'une seule fois.

```
# store_manager.py
if __name__ == "__main__":
    populate_redis_from_mysql()
    server = HTTPServer(("0.0.0.0", 5000), StoreManager)
    server.serve_forever()
```

```
# commands/write_order.py
def sync_all_orders_to_redis():
    r = get_redis_conn()
    orders_in_redis = r.keys("order:*")
    rows_added = 0
    try:
        if len(orders_in_redis) == 0:
            session = get_sqlalchemy_session()
            orders_from_mysql = session.query(Order).all()
            for order in orders_from_mysql:
                r.hset(f"order:{order.id}", mapping={"user_id": order.user_id, "total":
order.total_amount})
                for item in order.order_items:
                    r.incr(f"product:{item.product_id}", int(item.quantity))
            rows_added = len(orders_from_mysql)
            session.close()
        else:
            print('Redis already contains orders, no need to sync!')
    except Exception as e:
        print(e)
        return 0
    finally:
        return len(orders_in_redis) + rows_added
```

Question 2

Quelles méthodes avez-vous utilisées pour lire des données à partir de Redis ?

Pour lire les commandes depuis Redis, deux méthodes Redis ont été utilisées :

- `r.keys("order:*")` pour récupérer toutes les clés correspondant au pattern
- `r.hgetall(key)` pour lire les champs d'un hash (`user_id`, `total`)

```
# queries/read_order.py
def get_orders_from_redis(limit=9999):
    r = get_redis_conn()
    keys = r.keys("order:*")
    orders = []
    for key in keys[:limit]:
        data = r.hgetall(key)
        order_id = int(key.split(":")[1])
        orders.append(OrderRedis(
            id=order_id,
            user_id=int(data.get("user_id", 0)),
            total_amount=float(data.get("total", 0))
        ))
    orders.sort(key=lambda o: o.id, reverse=True)
    return orders
```

La vue `order_view.py` appelle désormais `list_orders_from_redis(10)` au lieu de `list_orders_from_mysql(10)`, ce qui fait que toutes les lectures de commandes passent exclusivement par Redis.

Question 3

Quelles méthodes avez-vous utilisées pour ajouter des données dans Redis ?

Pour insérer une commande dans Redis, la méthode `r.hset()` a été utilisée afin de stocker la commande sous forme de hash. Pour les articles (nécessaires au rapport des meilleures ventes), `r.incr()` a été utilisé pour incrémenter le compteur de chaque produit.

```
# commands/write_order.py
def add_order_to_redis(order_id, user_id, total_amount, items):
    r = get_redis_conn()
    r.hset(f"order:{order_id}", mapping={"user_id": user_id, "total": total_amount})
    for item in items:
        r.incr(f"product:{item['product_id']}", int(float(item['quantity'])))
```

Cette méthode est appelée dans `add_order()` immédiatement après le `session.commit()` de MySQL.

Question 4

Quelles méthodes avez-vous utilisées pour supprimer des données dans Redis ?

La méthode `r.delete()` a été utilisée pour supprimer la clé correspondant à la commande dans Redis.

```
# commands/write_order.py
def delete_order_from_redis(order_id):
```

```
r = get_redis_conn()
r.delete(f"order:{order_id}")
```

Cette méthode est appelée dans `delete_order()` après le `session.commit()` de MySQL, ce qui maintient la cohérence des données entre les deux bases.

Question 5

Si nous souhaitions créer un rapport sur les produits les plus vendus, les informations dans Redis sont-elles suffisantes ?

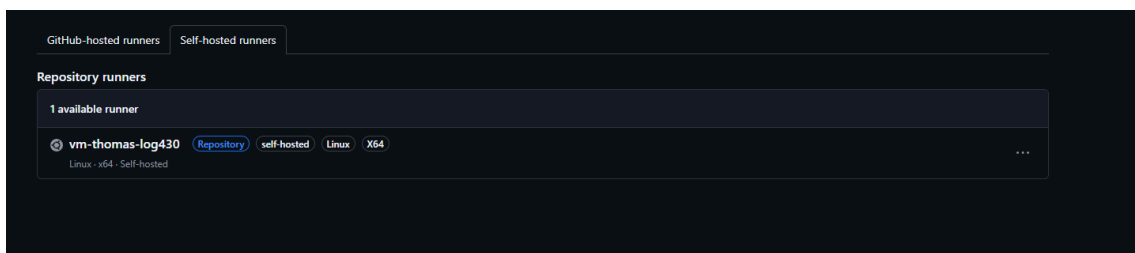
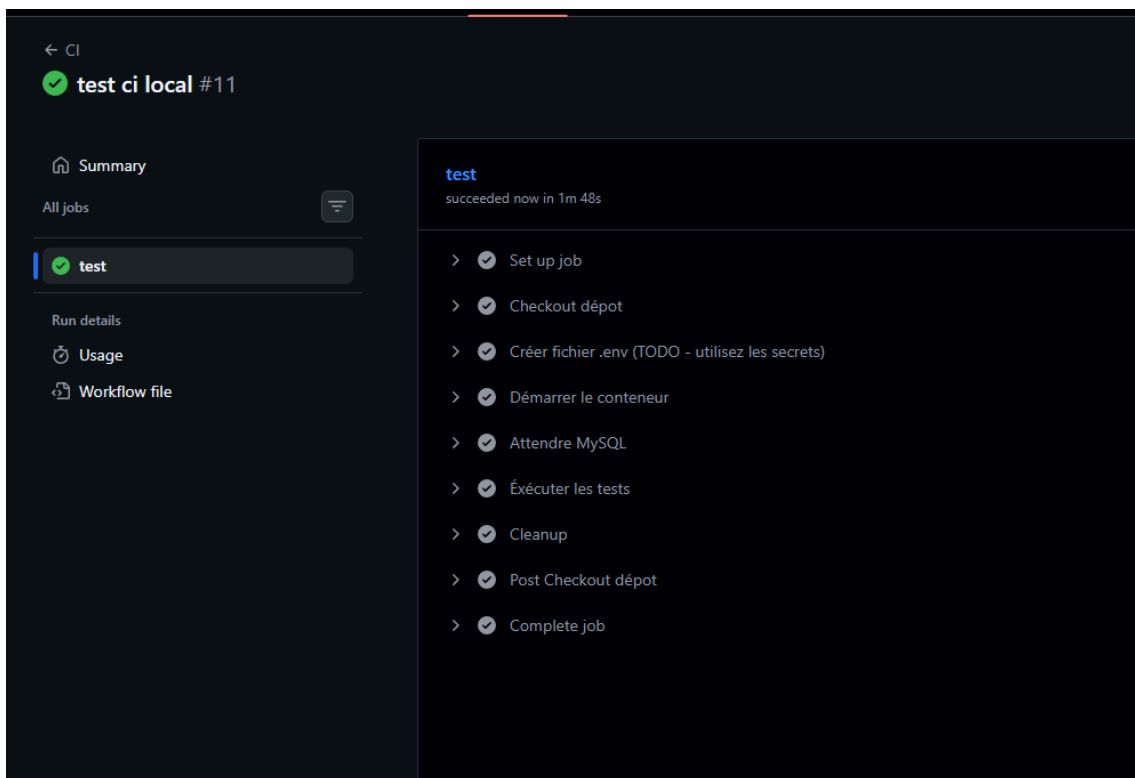
Non, les informations initiales dans Redis (uniquement `user_id` et `total` par commande) ne sont pas suffisantes pour générer ce rapport. Il faudrait également connaître quels produits ont été commandés et en quelle quantité.

Pour palier ce problème, nous avons ajouté dans Redis un compteur par produit, mis à jour à chaque nouvelle commande via `r.incr()`. Cela crée des clés de la forme `product:{id}` dont la valeur représente la quantité totale vendue.

```
# queries/read_order.py
def get_best_sellers():
    r = get_redis_conn()
    keys = r.keys("product:*")
    products = []
    for key in keys:
        product_id = int(key.split(":")[1])
        quantity = int(r.get(key) or 0)
        products.append((product_id, quantity))
    return sorted(products, key=lambda item: item[1], reverse=True)
```

Avec cette approche, le rapport des articles les plus vendus est généré entièrement depuis Redis sans requête MySQL.

Annexe A – CI et Self-Hosted Runner



Annexe B – Tests

